

Production Ready

Phase 3: Advanced AI Development Features

Persistent, organized, and efficient AI-assisted development workflows

5

Core Features

88.6%

Test Coverage

305+

Tests

4,903



Session Management

Organize your work into focused development sessions with specific modes and lifecycle tracking.

- ✓ 6 session modes: Planning, Building, Testing, Refactoring, Debugging, Deployment
- ✓ Full lifecycle: Create → Start → Pause → Resume → Complete/Fail
- ✓ Project association and tagging
- ✓ Session queries and filtering
- ✓ Export/import for sharing

```
// Create and start a session
sess := sessionMgr.Create("implement-auth", session.ModeBuilding, "
sess.AddTag("authentication")
sessionMgr.Start(sess.ID)

// Work happens here...

sessionMgr.Complete(sess.ID)
```

Test Coverage: 90.2% (83 test cases)



Memory System

Maintain conversation context with intelligent message and conversation management.

- ✓ Role-based messaging (User, Assistant, System)
- ✓ Conversation grouping and organization
- ✓ Token counting and tracking
- ✓ Automatic trimming with limits
- ✓ Search and filtering
- ✓ Export/import with snapshots

```
// Create conversation
conv := memoryMgr.CreateConversation("Feature Implementation")
conv.SessionID = sess.ID

// Add messages
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage("Help me implement"))
memoryMgr.AddMessage(conv.ID, memory.NewAssistantMessage("Sure, let me help you")

// Manage size
conv.SetMaxMessages(500)
conv.SetMaxTokens(50000)
```

Test Coverage: 92.0% (50+ test cases)



State Persistence

Never lose your work with automatic state saving and recovery.

- ✓ Auto-save with configurable intervals
- ✓ 3 formats: JSON, Compact JSON, JSON+GZIP
- ✓ Atomic writes for data integrity
- ✓ Backup and restore functionality
- ✓ Concurrent-safe operations
- ✓ Event callbacks for monitoring

```
// Initialize persistence
store := persistence.NewStore("./helixcode_data")
store.SetSessionManager(sessionMgr)
store.SetMemoryManager(memoryMgr)
store.EnableAutoSave(300) // 5 minutes

// Load previous state
store.Load()

// Create backup
store.Backup("./backups/backup-20250107")
```

Test Coverage: 78.8% (40+ test cases)

Template System

Accelerate development with reusable templates for code, prompts, and workflows.

- ✓ 6 types: Code, Prompt, Workflow, Documentation, Email, Custom
- ✓ Variable substitution with {{placeholder}} syntax
- ✓ 5 built-in production templates
- ✓ Search and filtering
- ✓ File I/O for template libraries
- ✓ Export/import for sharing

```
// Use built-in template
result, _ := templateMgr.RenderByName("Function", map[string]interface{}{
    "function_name": "ProcessData",
    "parameters":    "data []byte",
    "return_type":   "error",
    "body":          "return nil",
})

// Create custom template
tpl := template.NewTemplate("API Handler", "HTTP handler", templateMgr)
tpl.SetContent(`func Handle{{name}}(w http.ResponseWriter, r *http.Request) {
```

```
    {{implementation}}  
  }` )
```

Test Coverage: 92.1% (63 test cases)

Complete Integration

All systems work together seamlessly for powerful workflows.

```
func implementFeature(name string) {  
    // 1. Create session  
    sess := sessionMgr.Create(name, session.ModeBuilding, "project"  
    sessionMgr.Start(sess.ID)  
  
    // 2. Create conversation  
    conv := memoryMgr.CreateConversation("Implementation: " + name)  
    conv.SessionID = sess.ID  
  
    // 3. Use template  
    prompt, _ := templateMgr.RenderByName("Code Review", vars)  
  
    // 4. Add to conversation  
    memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))  
  
    // 5. Auto-saved by persistence  
    sessionMgr.Complete(sess.ID)  
}
```



Resources

- ✓ [Complete API Reference](#)
- ✓ [Quick Reference Guide](#)
- ✓ [Video Course \(12 videos\)](#)
- ✓ [Example Projects](#)
- ✓ [User Manual](#)

View API Reference

Quick Reference